

Lab 3: Component Modelling & Architectural Pattern Selection

Justification

NAME : SANTOSH N

SRN : PES1UG24CS615

Architecture Selection

We chose Layered Architecture for the Student Club Event Ticketing & Budget Portal System.

The system is split into three layers: a Presentation layer (Web UI Component), a Business layer (Event Proposal Manager, Budget Approval Service, QR Ticket Service, Notification Service) and a Data layer (Database Component). Components communicate only through defined provided/required interfaces (e.g. IProposalMgmt, IApprovalWorkflow, ITicketing, IProposalData). The Event Proposal Manager plays the role of the handout's Order Manager, and the Budget Approval Service plays the role of its Payment Service.

Two Reasons for the Choice

- **1. Layers match the approval workflow.** Club Leads and Campus Admin only interact through the UI, while the rule that a budget request must pass Faculty Coordinator → Finance Office → Dean (FR-001) lives in one place, the Business layer. Because every request must pass through it, no request can reach the Dean without Finance sign-off, and the UI can change without touching approval logic.
- **2. Right-sized for a campus system with strong consistency needs.** The portal serves one university with moderate load, so Microservices would add service discovery, network hops and distributed transactions without a real benefit. A pure Client-Server design would mix UI and workflow logic. A layered design with one shared database keeps budgets, approvals and tickets consistent (no double allocation or duplicate tickets) and is simple for a student team to build and maintain.

Security Advantage

Only the Business layer can reach the Database Component, so the UI has no direct data access. Role-based access control (Club Lead, Faculty Coordinator, Finance, Dean, Admin) and approval checks are enforced in one central place, which also makes audit logging of every sign-off straightforward. QR tickets carry a server-signed token, and the QR Ticket Service validates it server-side, so forged or tampered QR codes are rejected at checkin.

Performance Benefit

The critical path is short: at check-in, the scanner goes Web UI → QR Ticket Service → Database (one indexed lookup by ticket ID). There are no calls between other business services, which helps meet NFR-001 (QR validation under 100 ms). The Ticket Service can also cache recently issued and validated ticket records, and because each layer is stateless, extra instances can be added behind a load balancer during peak ticket-release or event-day check-in.

Trade-off Acknowledged

Layering adds slight call overhead and couples layers to some degree, and a shared database is a scaling limit. For this scale these costs are small, and they can be reduced later by moving the QR Ticket Service into its own service if traffic grows.